

ENGG 200 W26

Solution Code Guide



Introduction

This document will walk through and explain the provided sample code, specifically by telling you which areas of the code should be left alone, and which areas can be modified to add or change the functionality of your ENGG 200 boat project.

On D2L under “Project Documents” you can download the two provided MicroPython files. These are:

- *transmitter_main.py*
- *receiver_main.py*

These files will hereafter be referred to as “Transmitter” or “Remote” interchangeably, and “Receiver” or “Boat” interchangeably. You **MUST** rename both of these files to “main.py” when saving them to your Raspberry Pi Pico so that they run automatically when the Pico receives power.

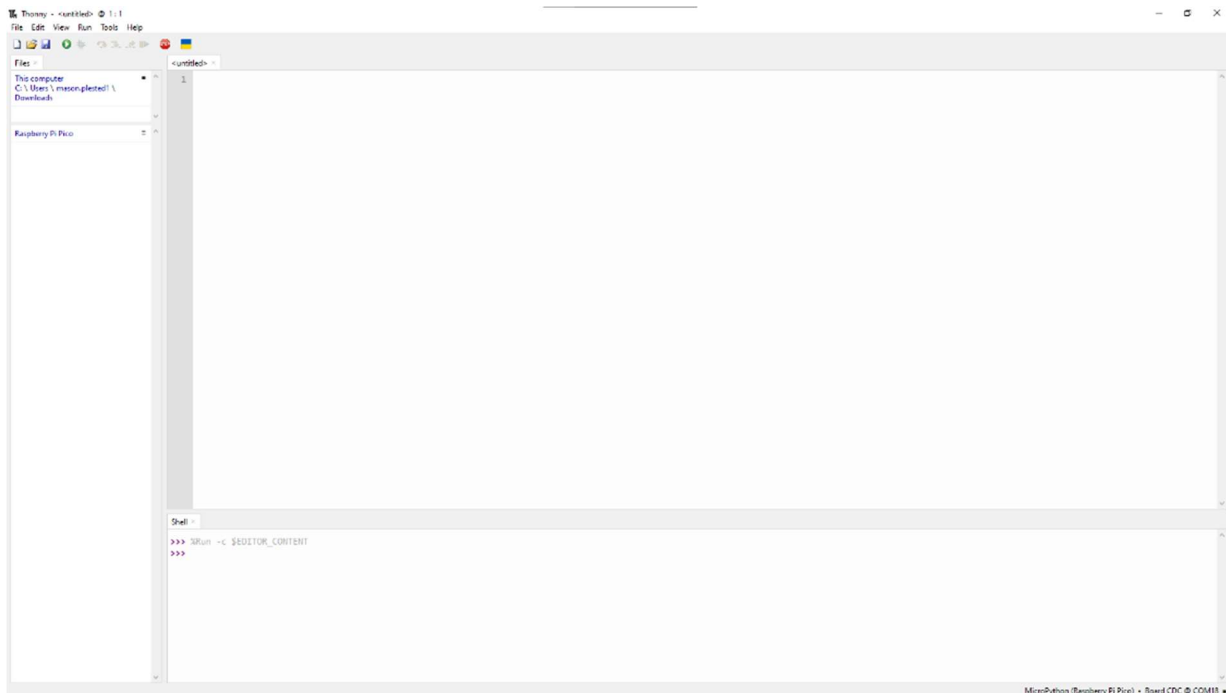
The next page will explain how to upload and save files onto your Raspberry Pi Pico from your computer. We are assuming that you have already installed MicroPython on your Pi Pico. If not, see *Thonny_Install guide* or *Pi Pico Setup* video on D2L under “Project Documents.”

Copying files onto your Raspberry Pi Pico

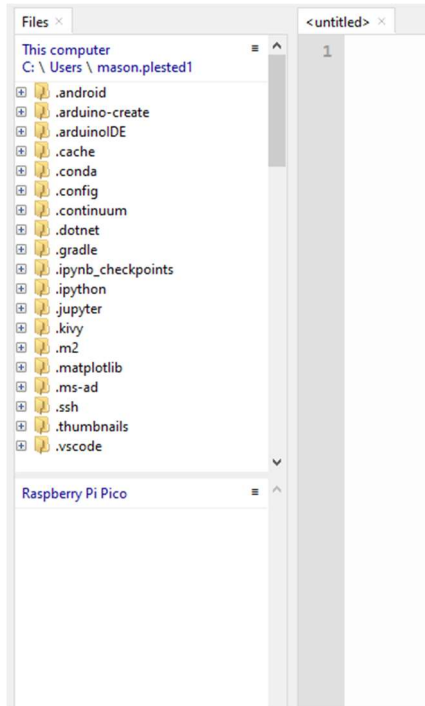
Some components we use with the Raspberry Pi Pico require specific code to function correctly. Typically, these are called libraries or modules. In ENGG 200 Winter 2026, you have been given code which will link two Raspberry Pi Pico devices with a Bluetooth connection, allowing one to control the other. **This is not a library**, however, the process to upload it to our Pico is the same.

First, we need to make sure that our Pi Pico is connected to our computer and connected in Thonny. If you are unsure of how to do this, please review the *Thonny_Install* guide or *Pi Pico Setup* video on D2L under “Project Documents.”

Once you have this done, your Thonny should look something like this:



If you do not see the Files pane on the left side, enable it by hovering over “View” in the top navigation bar, and then selecting “Files.” Once you can see your files, you may notice a section that says “This computer” above where it shows the files currently on your Pi Pico. Using this, we can copy over any files we want so long as they fit within the storage limits of the board. We can also drag the grey bar between “This computer” and “Raspberry Pi Pico” to make it easier to see what we have on our computer. After resizing, you may see something like this:



This method of viewing the files on your computer is called **Tree View**. To open folders, you can either press the plus button or double click the folder you want to enter. To determine where you are in your files, you can look at the path listed underneath “This computer.” This is also how you return to a higher-level folder: hover over a previous folder name and click to return to that level.

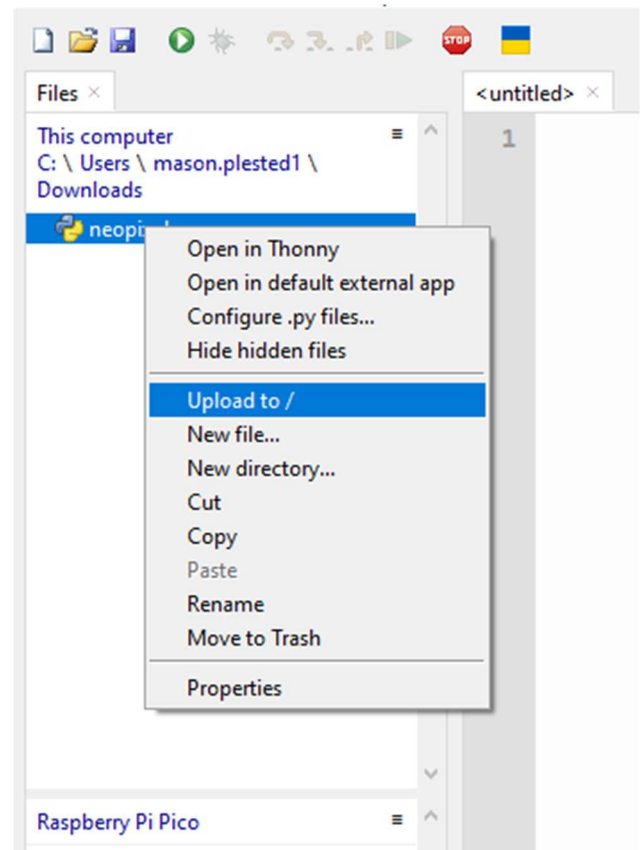
We can navigate to where we downloaded the provided code file and view it in the explorer. In my case, this is the Downloads folder. Once you have found the file you want to copy over, you can right click the file name and select

Upload to /

Once you select the option to upload the file, you should see it appear under the “Raspberry Pi Pico” section of the file viewer. If the file you uploaded was a MicroPython library file, you can open it in Thonny to view it if you want. Image and audio files cannot be directly viewed without first downloading them to your computer from the Pi Pico first.

Another option for copying .py files specifically, is to open them in Thonny using “Open” under “File” in the top navbar and then saving the file to your Pi Pico the same way you save files normally. This will not work for image, audio, and other non-python files.

REMEMBER: YOU MUST RENAME THE PROVIDED FILES TO “MAIN.PY” OR YOUR CODE WILL NOT RUN AUTOMATICALLY WHEN THE RASPBERRY PI PICO RECIEVES POWER.

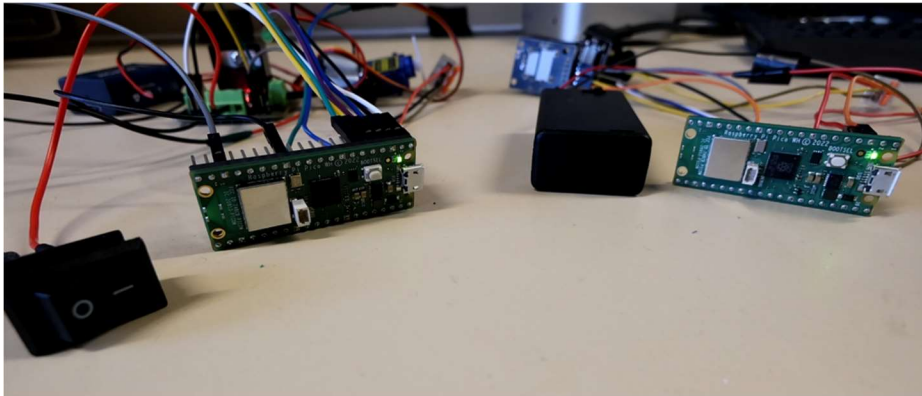


Using the provided code (Basics)

The provided code is intended as a starting point that will be enough to get you most of the way to a final implementation. This code will use Bluetooth to transmit the position of two joysticks on the controller to the boat, which will decode and convert it into a speed and direction for each of the provided DC motors. When the transmitter code is saved to your remote controller Pi Pico, and the receiver code is saved to your boat Pi Pico with both renamed exactly as ***main.py***, you should be able to provide power to each of the Pi Picos and have them automatically connect. This connection will be signaled by blinking the onboard LED. When the Pi Picos are pairing, they will blink quickly. When paired, they will blink slower to indicate they are connected.

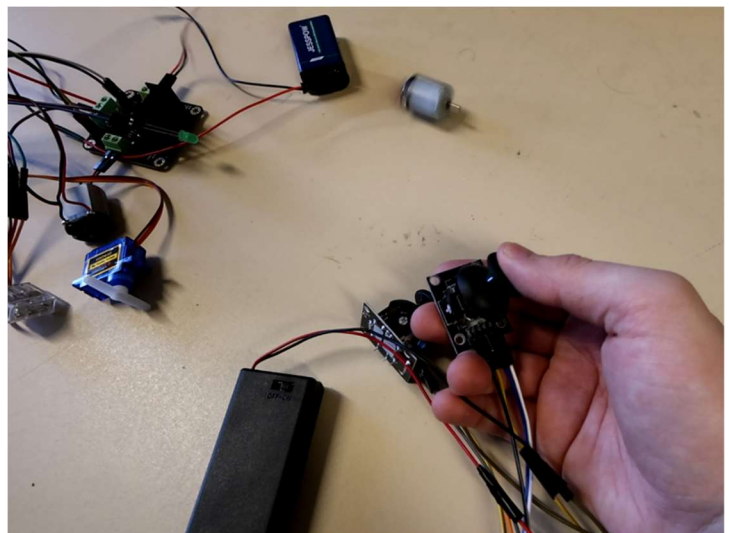
ONBOARD LED STATUS	MEANING
Static (Always off)	Pi Pico is not powered on.
Blinking (Fast)	Searching for Bluetooth connection.
Blinking (Slow)	Controller and boat are connected.

The Picos may not blink in sync, and that is not an issue as long as they are both blinking at the same speed.



Seen here are two Picos set up to follow the example wiring diagram provided on D2L. If one Pico loses a connection with the other, it should automatically return to “searching” mode after a few seconds where it will be able to re-establish the connection if possible.

Assuming your wiring is correct, you should now be able to move the joystick on the controller and have the motors on the boat spin forwards and backwards in relation to your movements:



To make your code function, assuming you have followed the example wiring diagram, you will need to do the following steps:

1. UPDATE YOUR DEVICE NAME

The most important first step is to change the line of code in **BOTH** the receiver and transmitter files that looks similar to this:

```
_DEVICE_NAME = "engg_200"
```

If you do not change this line to your block and group number (i.e. "B1_A4", "B8_C1"), you will encounter issues where your remote may control other groups boats, and their boats may receive input from your controller. You **MUST** change the value in the quotation marks, but not any of the other variables below it.

2. DO NOT CHANGE ANY OTHER CODE

If you do not need to make changes, DO NOT MAKE THEM. The sample code is not intended to be exhaustive; it is only enough to allow you to control two motors using two joysticks. If you want to implement other components, you will need to spend some time learning how each individual component works and then update the starter code. If you do want to add or modify behaviours, the section below "Updating the provided code (Advanced)" will provide a starting point.

3. ENSURE YOUR WIRING IS CORRECT

If your code does not behave as intended, your first troubleshooting step should be to ensure your wiring is correct. If you have a component plugged into a GPIO pin that differs from what is specified in the code, that component will not be controlled or may be controlled incorrectly.

4. MAKE SURE YOUR FILE NAMES ARE CORRECT

The sample code will not run automatically when your Pi Picos receive power unless you save the code to the Pi Pico with the name **main.py**. If you took ENDG 233 in the previous semester, you may already have a file named main.py on your Pi Pico. You will need to either delete or overwrite this file in order to observe the intended behaviour.

If you are unable to make the provided sample code function, first re-check each of the above steps. If that does not help, ask for assistance during your regularly scheduled Studio session.

Updating the provided code (Advanced)

If you want to modify or add functionality to the provided code, it is important to first understand what the provided code is doing, and what areas of the code are safe to modify. The provided code includes some concepts not covered in ENDG 233, including asynchronous code. When you power on the Pi Picos with the provided code, they will run the code in the section labeled “Main Loop”. This code will start other sections of the code which will attempt to pair the two Picos, and once the two Picos are connected the remote Pico will transmit a sentence to the boat Pico which will attempt to decode it. By default, that sentence will look like this:

```
msg = f"L:{left_val},R:{right_val}"
```

Analyzing this, we are transmitting ‘L:’ followed by a value, and then ‘R:’ followed by a value. By default, the value we transmit is how much the left and right joysticks are being moved forward or backwards. If we were to plug in our controller and have it connected, it would instantly begin transmitting a value close to this:

```
L:33320,R:32455
```

Why is the value we transmit for left and right approximately 33,000? We are reading in the current position of the joysticks using the two lines:

```
left_val = adc_left.read_u16()  
right_val = adc_right.read_u16()
```

These lines of code use the Pi Pico’s built in Analog to Digital (ADC) converter to read a variable voltage level as a number in the range 0 to 65,535. This range is because the analog voltage level from 0V to 3.3V is mapped as something called a 16-bit integer, which can store a maximum of 65,535 numbers. By default, the joystick’s resting position allows approximately 1.65V through, which gets converted to approximately the number 33,000 on the Pi Pico. If we move the joystick in one full direction, we might now let through 0.05V which is mapped to a very low number like 500, or the opposite full direction which will let through 3.3V which is mapped to nearly 65,000. This number then lets us determine where the joystick is currently positioned.

In the example above, the left and right joysticks do not read exactly the same when at rest. This is normal and expected. Manufacturing variance and limits with the Pi Pico itself mean that we cannot expect perfect accuracy. When we write our code, we need to keep this slight variance in mind to avoid undesired behaviour.

On the receiver, this sentence is handled in the function called *handle_command* located in the section labeled “Handle Joystick Data”. This code first preforms the following steps:

```
msg = cmd.decode().strip()
left_adc = int(msg.split(",")[0].split(":")[1])
right_adc = int(msg.split(",")[1].split(":")[1])
```

These lines attempt to read the message that was sent from the controller and split it apart, so we get the ADC values back in a raw form. In the case of the example above, this would give us a left value of 33320 and a right value of 32455 without the rest of the sentence. If the receiver cannot decode this sentence for whatever reason, it will stop the motors and prevent the code from going any further.

This section of code will need to be changed if you want to add or change what is being transmitted from the controller to the boat. On the controller end, you will need to add new code where the lines for reading the joysticks are located and use that new data to modify the sentence that is being transmitted. On the boat side, you will need to add a new line underneath the code to split the sentence apart so that we can properly read our new value. For example, if we wanted to add a button to the controller, we might modify the sentence to read:

L:33320,R:32455,B:0

Where “B:0” indicates that the button *is not* currently pressed, and “B:1” indicates that the button *is* currently pressed. On the boat side, we would add the following line beneath *right_adc*:

```
button_status = int(msg.split(",")[2].split(":")[1])
```

Which will allow us to check if *button_status* is either 0 or 1 and take some action accordingly.

Returning to the earlier example, after we have received and decoded the sentence from the controller, the code moves on to a function called *scale_adc*. This will convert the raw 16-bit integer from the joystick into a number in the range (-100, 100). Using this value, we call another function outside of the *handle_command* code called *set_motor*, located earlier in the file. This function spins one of the two motors at a specified speed. To make a motor more or less responsive, you can modify these values which control when the joystick input gets ignored (called “deadzone”) and the maximum allowed speed in the range (0, 100):

```
DEADZONE_LOW = -2
DEADZONE_HIGH = 2
MAX_SPEED = 100
```

If you wanted to add new functionality to the provided code, the recommended method would be to add new code into *handle_command* to get the values from the controller and then add a new function outside of the *handle_command* function to actually control the new feature. If you are looking for where to start with additional components or just want to test the individual components to

determine if there may be a fault with one, please use the provided “Component testing” code found on D2L. Understanding how the code works to control one component by itself is important, as attempting to integrate it directly into the provided code without a thorough understanding of what you are adding can lead to difficult debugging or complex unwanted behaviour.

In general, if you want to add new components you should take the following steps:

1. Run the component with the provided sample code.
2. Figure out what parts of the sample code are required for what behaviour you want.
3. Add the required pin definitions (if needed).
4. Add a function to control the component, separate from *handle_command* for easier debugging.
5. Call the new function from *handle_command* with the required information as a parameter.

For example, if we wanted to add two LED lights to the boat that showed which motor(s) were currently being controlled, we may want to view the *led_sample.py* file found on D2L. This file is fairly short, and the entire code is included here:

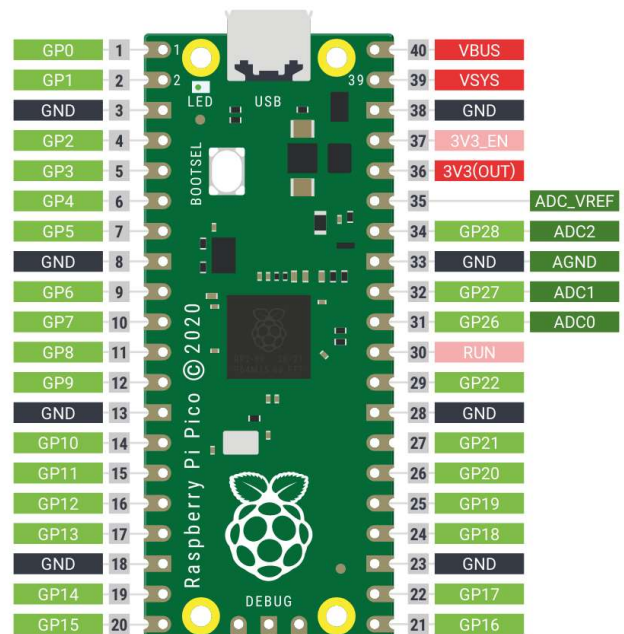
```
from machine import Pin
from time import sleep
```

```
# Set LED pin
led = Pin(2, Pin.OUT)
```

```
# Test LED
while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

This code takes a single LED light connected to GPIO Pin 2 and turns it on for one second, and then turns it off for one second, and repeats. If we wanted to add the LED lights to the provided code, we would want to add pin definitions for each light to the receiver code and add led on and off control to the *handle_command* function.

First, we need to make sure that our wiring is correct so that our pin definitions match. On the Raspberry Pi Pico, there are 40 physical pins but only 28 General Purpose Input / Output (GPIO) pins. Since our example wiring diagram for the boat already uses GPIO pin 2 for one of the provided LED lights, we can use that pin definition easily. For our second LED light, we would need to use a different GPIO pin that is not already in use. For example, we will use GPIO 17 since it is conveniently located beside a ground pin.



Now that we know which pins the LED lights are connected to, we can add the pin definitions to the receiver code. In the provided code, there is a section labeled “Motor Pins (L298N)”. For the sake of organization, we will add a new section below called “LED Pins” which looks like this:

```
# =====  
#           LED PINS  
# =====  
  
LED_left = Pin(2, Pin.OUT)    # Left motor status LED  
LED_right = Pin(17, Pin.OUT)  # Right motor status LED
```

Now our code knows which GPIO pins control each of the LED lights. Make sure that when you assemble your boat, the LED physically on the left is assigned to *LED_left*, and vice versa. Under the section labeled “Motor Control”, we can add a new function for controlling the LED lights:

```
def toggle_lights(left_speed, right_speed):  
    if left_speed > DEADZONE_HIGH or left_speed < DEADZONE_LOW:  
        LED_left.on()  
    else:  
        LED_left.off()  
  
    if right_speed > DEADZONE_HIGH or right_speed < DEADZONE_LOW:  
        LED_right.on()  
    else:  
        LED_right.off()
```

Finally, we add a new line beneath where we call to this function in *handle_command* beneath where we set the motors:

```
toggle_lights(left_speed, right_speed)
```

Now, when either motor is spinning, its respective LED light should also turn on. This can be useful as a status indicator to visually determine if your boat is behaving as you intend, even if you cannot directly see the output of the motors. Note this this *is not* the only way to add this functionality, but it is recommended to do it like this to make debugging easier, rather than trying to force everything into one large function that can be more difficult to debug.

Having your components work in exactly the way you expect can take time. It can help to write out your expected behaviours and how they should relate inputs to outputs, as well as the required logic to make them happen. Always make sure to test components individually before attempting to integrate them, which will allow you to determine if a component may be faulty before going further. If you encounter any issues, please review this guide and attempt to solve the issue yourself, and then ask for help in your regular Studio session (Do not attend other sessions, allow those students to get help!)